

Appendix A: Equilibrium Solver

A1. The Positive Continuous Fraction (PCF) Method

A1.1 Overview and Motivation

Solving chemical equilibrium problems requires finding species concentrations that simultaneously satisfy mass action laws and mass balance equations (conservation of mass). Traditional Newton–Raphson methods iteratively solve $\Delta C = -J^{-1}f$ where C is the vector of component concentrations, f is the vector of mass balance residuals, J is the Jacobian matrix with entries $J_{ij} = \frac{\partial f_i}{\partial C_j}$ (the partial derivative of the i -th residual with respect to the j -th component), and ΔC is the correction applied at each iteration (Tadanier and Eick 2002). This requires a matrix inversion at every step, which is computationally expensive for large systems and the method can fail when the initial guess is poor. The Positive Continuous Fraction (PCF) method avoids this entirely. It is a derivative-free fixed-point iteration that works in logarithmic space, which naturally keeps all concentrations positive. At each step, it splits the mass balance into positive and negative contributions and computes a correction factor from their ratio. PCF converges linearly on its own, but can be sped up significantly when combined with Anderson Acceleration (Section A5).

A1.2 Mathematical Foundation

The mathematical framework for the PCF method (Section A4) and Anderson Acceleration (Section A5) follows (Awada et al. 2025). All equations are adapted to the notation used in this appendix.

Consider a system with N_c components, N_r reactions and N_s species (concentrations that can be expressed as a function of the components and the equilibrium reactions). The stoichiometric matrix s has dimensions $N_s \times N_c$. Each entry s_{ij} gives the stoichiometric coefficient of component j in the formation reaction for species i . The log-space variable ω_j represents $\log_{10}(C_j)$, the base-10 logarithm of the component concentration.

The mass action law states that for each secondary species i , its concentration equals the equilibrium constant multiplied by the product of component concentrations raised to their stoichiometric powers:

$$C_i = K_i \prod_j C_j^{s_{ij}}$$

Where K_i is the effective equilibrium constant associated with the formation of species i . Taking the base-10 logarithm of both sides allows us to convert the multiplication to addition. Using the logarithm product rule $\log(ab) = \log(a) + \log(b)$:

$$\log_{10}(C) = \log_{10}(K_i) + \log_{10}\left(\prod_j C_j^{s_{ij}}\right)$$

Now applying the logarithm power rule to each term in the product:

$$\log_{10}(C_i) = \log_{10}(K_i) + \sum_{j=1}^{N_s} s_{ij} \omega_j$$

where defining $\omega_j = \log_{10}(C_j)$

Or in compact vector form:

$$\log_{10}(C) = \log K + s \omega$$

Conservation of mass requires that the total analytical concentration of component j equals the sum of that component's contribution across all species. Each species i contributes s_{ij} moles of component j per mole of species i. Therefore, the mass balance constraint is:

$$T_j = \sum_{i=1}^{N_s} s_{ij} C_i$$

Here T_j is the known total concentration of component j (an input to the system).

A1.3 The Five PCF Steps

Starting from an initial guess, PCF repeats five steps each iteration until convergence:

Step A: Define ω_j for each component as the current estimate of $\log_{10}(C_j)$.

Step B: Compute all species concentrations via the mass action law. For each species i , evaluate:

Starting from the log-space mass action law derived above:

$$\log_{10}(C_i) = \log_{10}(K_i) + \sum_{j=1}^{N_s} s_{ij} \omega_j$$

Step C: Compute the reactive sum and the product sum for each component. The PCF method splits the mass balance for each component j into two sums of positive quantities, called the reactive sum R_j and the product sum P_j . We derive this splitting below.

The mass balance for component j relates the total concentration T_j (a known quantity) to the species concentrations C_i through the stoichiometric matrix s :

$$T_j = \sum_{i=1}^{N_s} s_{ij} C_i$$

Each stoichiometric coefficient s_{ij} is either positive, negative, or zero. We separate the species into two groups based on the sign of s_{ij} . Species with $s_{ij} > 0$ contribute positive terms to the sum. Species with $s_{ij} < 0$ contribute negative terms. Since concentrations C_i are always positive, a term $s_{ij} C_i$ with $s_{ij} < 0$ is negative. We write it as $|s_{ij}| C_i$ so that we work only with positive quantities. Splitting the sum gives:

$$T_j = \sum_{i=1, s_{ij}>0}^{N_s} s_{ij} \cdot C_i - \sum_{i=1, s_{ij}<0}^{N_s} |s_{ij}| \cdot C_i$$

To form two sums that are both strictly positive, we rearrange this equation. The total concentration T_j can be positive, negative, or zero depending on the chemistry.

$$T_j + \sum_{i, s_{ij}<0}^{N_s} |s_{ij}| \cdot C_i = \sum_{i, s_{ij}>0}^{N_s} s_{ij} \cdot C_i$$

We define the left-hand side as the product sum P_j and the right-hand side as the reactive sum R_j :

$$P_j = T_j + \sum_{i, s_{ij}<0}^{N_s} |s_{ij}| \cdot C_i \quad (\text{product sum, when } T_j \geq 0)$$

$$R_j = \sum_{i, s_{ij}>0}^{N_s} s_{ij} \cdot C_i \quad (\text{reactive sum, when } T_j \geq 0)$$

The naming follows Awada et al. (2025): the reactive sum R_j collects the positive-coefficient (reactant) terms, and the product sum P_j collects the absolute-value negative-coefficient (product) terms. The total concentration T_j (or $|T_j|$) is placed on whichever side needs it to remain positive.

At equilibrium, the mass balance is exactly satisfied, which means $P_j = R_j$. Away from equilibrium, the ratio $\frac{P_j}{R_j} \neq 1$, and it measures how far the system is from balance for component j . The PCF iteration (Step D) uses $\log_{10}\left(\frac{P_j}{R_j}\right)$ to compute the correction to ω_j .

Step D: Compute the PCF correction for each component. The update is derived from the ratio of the product sum over the reactive sum:

$$\delta_j = \frac{1}{\nu_{\min,j}} \log_{10} \left(\frac{P_j}{R_j} \right)$$

We now derive the correction $\delta_j = \log_{10} \left(\frac{P_j}{R_j} \right)$, where $\nu_{\min,j}$ is a stoichiometric scaling factor (defined below). From Step C we have R_j (reactive sum) and P_j (product sum) for each component j . At chemical equilibrium, the mass balance is satisfied, which requires:

$$P_j = R_j \quad (\text{at equilibrium})$$

When the system is not yet at equilibrium, $P_j \neq R_j$. Their ratio $\frac{P_j}{R_j}$ measures imbalance: if greater than 1, the component concentration must increase; if less than 1, it must decrease. Since our unknown $\omega_j = \log_{10}(C_j)$, the correction for ω_j is simply:

$$\Delta = \log_{10} \left(\frac{P_j}{R_j} \right)$$

This has a key property: at equilibrium $P_j = R_j$, so the ratio is 1 and $\log_{10}(1) = 0$, meaning the correction vanishes automatically. The sign also works correctly: $\frac{P_j}{R_j} > 1$ gives a positive correction (increase ω_j), and $\frac{P_j}{R_j} < 1$ gives a negative correction (decrease ω_j). However, we need one more scaling step. Recall that $\omega_j = \log_{10}(C_j)$, so changing ω_j by an amount Δ gives:

$$C_j \rightarrow 10^{\omega_j + \Delta} = C_j \cdot 10^\Delta$$

Each secondary species i depends on component j through the stoichiometric power s_{ij} in the mass action law. So, a change Δ in ω_j changes species i by the factor:

$$C_i \rightarrow C_i \cdot 10^{s_{ij} \cdot \Delta}$$

If $|s_{ij}| > 1$, a unit change in ω_j causes a larger-than-unit change in $\log_{10}(C_i)$, which can overshoot equilibrium. To prevent this, we divide the raw correction by the smallest nonzero $|s_{ij}|$ in column j , considering all species that participate in the mass balance for component j (both those contributing to R_j and those contributing to P_j , i.e., both positive and negative stoichiometric coefficients taken in absolute value). We call this $\nu_{\min,j}$:

$$\nu_{\min,j} = \min\{ |s_{ij}| : s_{ij} \neq 0 \}$$

Dividing by $\nu_{\min,j}$ ensures the most sensitive species sees a correction no larger than the raw log-ratio. Combining the log-ratio with this stoichiometric scaling gives the PCF correction:

$$\delta_j = \frac{1}{\nu_{\min,j}} \log_{10} \left(\frac{P_j}{R_j} \right)$$

In our carbonate system all nonzero stoichiometric entries have magnitude 1, so $\nu_{\min,j} = 1$ and the correction simplifies to $\delta_j = \log_{10} \left(\frac{P_j}{R_j} \right)$.

Step E: Update: $\omega_j^{\text{new}} = \omega_j + \delta_j$. Repeating from Step B until the largest correction $\max |\delta_j|$ is smaller than the tolerance.

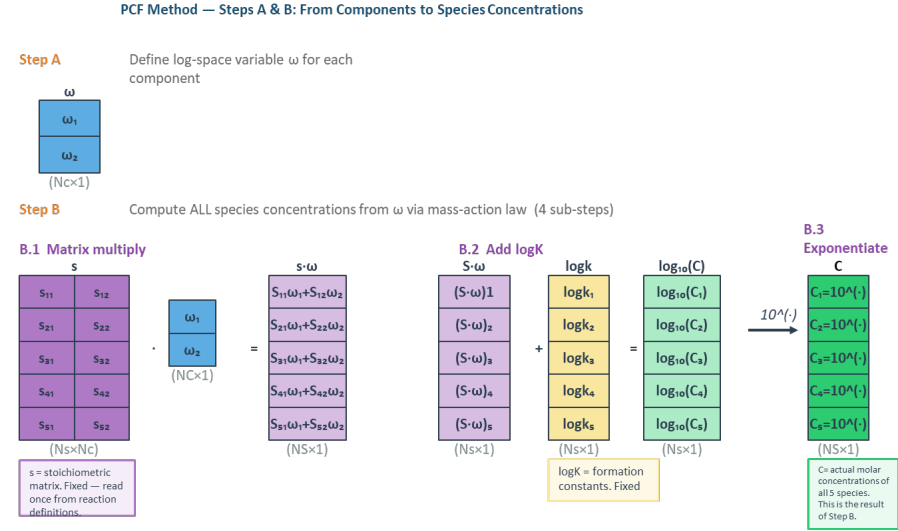


Figure A1. PCF Method, Steps A & B: From Components to Species Concentrations: ω ($N_c \times 1$): $\log_{10}$ of component activities which are the unknowns that the solver adjusts each iteration. s ($N_s \times N_c$): stoichiometric matrix or how species are built from components. $\log_{10} K$ ($N_s \times 1$): formation constants. C ($N_s \times 1$): species concentrations, computed from ω each iteration. For the carbonate example: $N_c = 2$ components (H^+ , HCO_3^-), $N_s = 5$ species (H_2CO_3 , CO_3^{2-} , H^+ , HCO_3^- , OH^-), $N_r = 3$ reactions. Complete Step B formula: $\log_{10}(C_i) = \sum_j s_{ij} \cdot \omega_j + \log_{10} K_i$, then $C_i = 10^{\log_{10}(C_i)}$. Each species concentration is determined by the component activities (ω) weighted by stoichiometric coefficients (s), shifted by the formation constant ($\log_{10} K$).

Step C

C.1 Multiply $s^T \cdot c$

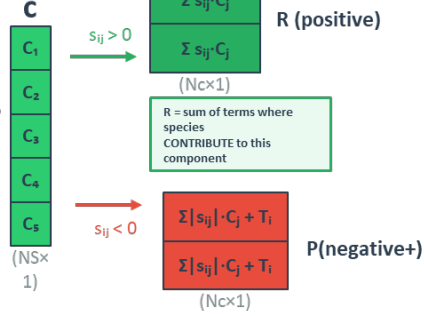
$$s^T = \begin{bmatrix} s_{11} & s_{21} & s_{31} & s_{41} & s_{51} \\ s_{12} & s_{22} & s_{32} & s_{42} & s_{52} \end{bmatrix}$$

(Nc×Ns)

s^T = transpose of S . Each ROW of s^T corresponds to one component.

Split mass balance $s^T \cdot C$ into positive contributions (R) and negative contributions (P)

C.2 Split by sign of s_{ij}



Step D

Compute correction δ , the key innovation: NO Jacobian needed!



Step E

Update ω and check convergence

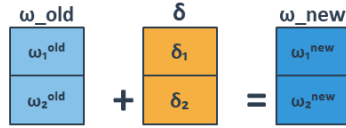


Figure A2. PCF Method, Steps C, D, E: Correction and Update: Step C: s^T = transpose of s . Each row of s^T corresponds to one component. R = sum of terms where species contribute to this component (positive stoichiometric coefficients). P = sum of |negative| terms + T_i . T_i = total concentration of component i (given as input). Step D: $\delta_i = \log_{10}(P_i/R_i)$, element-wise for each component.

A2. Anderson Acceleration (AA)

The PCF method converges reliably but slowly (linearly), each step roughly halving the error. To reach a tolerance of 10^{-10} could require hundreds of steps. Anderson Acceleration dramatically speeds this up by looking at the recent history of corrections and finding a smarter combination that jumps closer to the answer. Anderson Acceleration wraps around the PCF iteration from Section A5. At each AA iteration, five steps are applied in the following order: Step 1 runs one PCF cycle to produce $\delta^{(k)}$; Step 2 builds history matrices from past corrections and iterates; Step 3 solves for optimal weights via QR decomposition; Step 4 checks numerical stability; Step 5 computes the accelerated update and repeats. Each step is derived in the subsections below.

A2.1 AA Step 1: Run One PCF Cycle

At each iteration k , PCF gives us a correction vector $\delta^{(k)}$ and an updated guess $\omega_{\text{PCF}}^{(k)} = \omega^{(k)} + \delta^{(k)}$. Instead of blindly accepting this PCF step, Anderson Acceleration looks at the last few corrections and solves a small least-squares problem to identify an optimal weighted combination of past steps to improve the next solution update.

At each PCF iteration k , we compute a correction vector $\delta^{(k)}$. AA stores the DIFFERENCES between consecutive δ vectors:

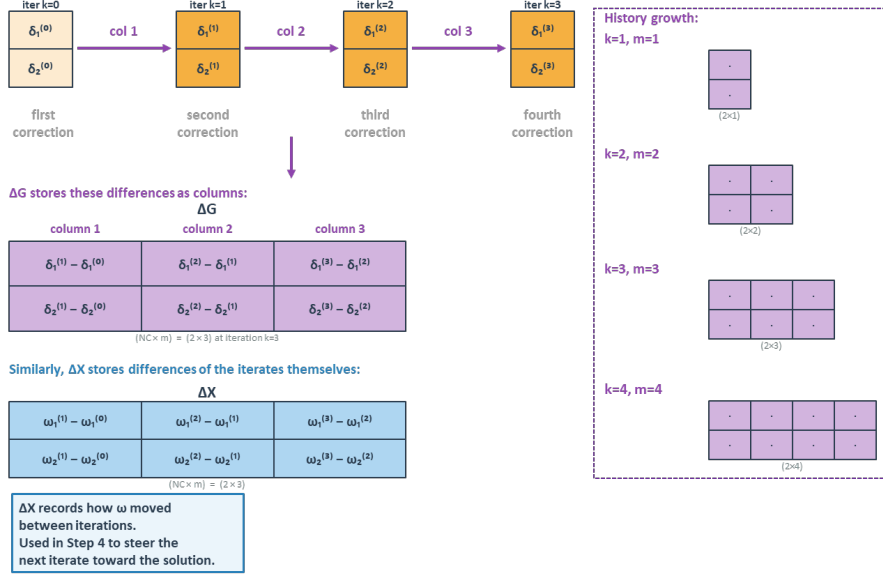


Figure A3. Anderson Acceleration, Step 1: Building the History Matrix ΔG : Each column of ΔG shows how the correction changed between two consecutive iterations. Each row = one component (row 1 = H^+ , row 2 = HCO_3^-). Column 1 is the oldest difference; column m is the newest. Number of columns m grows each iteration (up to m_{max}). If a column is nearly zero means that the correction barely changed and again means that direction is uninformative. ΔX records how ω moved between iterations and is used in Step 4 to steer the next iterate toward the solution.

A2.2 AA Step 2: Build History Matrices ΔG and ΔX

To see how Anderson Acceleration works concretely, imagine we are at iteration $k = 2$ in our 2-component system. We have corrections from iterations 0, 1, and 2. AA builds two matrices:

The first matrix, called ΔG , records how the correction vector has changed between iterations. Each column is the difference between the current correction and a previous one:

$$\Delta\delta^{(i)} = \delta^{(i+1)} - \delta^{(i)}$$

Each column captures a “change in correction.” If the correction barely changed between two iterations, that column will be nearly zero. If it changed a lot, that column carries useful information about the direction toward the solution.

Similarly, ΔX records how the iterate itself changed:

$$\Delta\omega^{(i)} = \omega^{(i+1)} - \omega^{(i)}$$

For our 2-component system, ΔG and ΔX are each 2-row matrices. The number of columns grows as iterations accumulate (up to a maximum depth $m = 4$, chosen based on numerical experiments rather than the number of components). The depth m is not linked to the number of components N_c ; it is a purely algorithmic parameter that controls how many past iterations AA uses to extrapolate. (Awada et al. 2025) tested $m = 2, 3, 4, 5$ and found that $m = 3$ or 4 gives the best trade-off between convergence speed and numerical stability. Larger m provides more history for AA to find a better acceleration direction but also increases the risk of ill-conditioning in the least-squares problem.

A2.3 AA Step 3: Solve for Optimal Weights via QR

The goal of Anderson Acceleration is to find a weight vector γ (gamma) that solves the least-squares problem:

$$\min_{\gamma} \|\delta^{(k)} - \Delta G \cdot \gamma\|$$

We now derive where this objective comes from and how to solve it. The correction $\delta^{(k)}$ is what PCF wants to add to $\omega^{(k)}$. If the solver is converging, the current correction should be predictable from how corrections changed in the past. We write this as a weighted sum of the columns of ΔG (the correction-change history from Step 2):

$$\delta^{(k)} \approx \gamma_1 \cdot \Delta\delta^{(k-m)} + \gamma_2 \cdot \Delta\delta^{(k-m+1)} + \dots + \gamma_m \cdot \Delta\delta^{(k-1)}$$

In matrix notation, this is simply:

$$\delta^{(k)} \approx \Delta G \cdot \gamma$$

The best γ minimizes the mismatch $\|\delta^{(k)} - \Delta G \cdot \gamma\|^2$. We use the squared norm (not the norm itself) because a smooth quadratic has a unique minimum found by standard linear algebra.

To solve this minimization, we factor ΔG into two simpler matrices using QR decomposition:

$$\Delta G = Q \cdot R$$

Here Q is orthogonal (its columns are perpendicular unit vectors, so $Q^\top Q = I$) and R is upper triangular (all entries below the diagonal are zero). For $m = 3$ history columns in our 2-component system, R looks like:

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ 0 & r_{22} & r_{23} \\ 0 & 0 & r_{33} \end{bmatrix}$$

Now substitute $\Delta G = QR$ into the objective and simplify step by step. Start:

$$\|\delta^{(k)} - \Delta G \cdot \gamma\|^2 = \|\delta^{(k)} - QR\gamma\|^2$$

Multiplying a vector by $Q^\top$ does not change its length (because $Q^\top Q = I$). So we multiply the vector inside the norm by $Q^\top$:

$$= \|Q^\top(\delta^{(k)} - QR\gamma)\|^2$$

Distribute $Q^\top$ into the parentheses:

$$= \|Q^\top \delta^{(k)} - Q^\top QR\gamma\|^2$$

Use $Q^\top Q = I$ to cancel:

$$Q^\top Q = I \quad \Rightarrow \quad Q^\top QR\gamma = R\gamma$$

So the objective simplifies to:

$$= \|Q^\top \delta^{(k)} - R\gamma\|^2$$

Define the vector $b = Q^\top \cdot \delta^{(k)}$ (one matrix-vector multiply). The problem is now:

$$\min_{\gamma} \|b - R\gamma\|^2$$

Since R has nonzero diagonal entries, we can make $\|b - R\gamma\|^2 = 0$ by setting:

$$R\gamma = b$$

This is an upper-triangular system. We solve it by back-substitution, starting from the last row. For $m = 3$:

$$\begin{bmatrix} r_{11} & r_{12} & r_{13} \\ 0 & r_{22} & r_{23} \\ 0 & 0 & r_{33} \end{bmatrix} \begin{bmatrix} \gamma_1 \\ \gamma_2 \\ \gamma_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Row 3 (bottom): $r_{33} \cdot \gamma_3 = b_3$, so:

$$\gamma_3 = \frac{b_3}{r_{33}}$$

Row 2: $r_{22} \cdot \gamma_2 + r_{23} \cdot \gamma_3 = b_2$. We already know γ_3 , so move $r_{23} \cdot \gamma_3$ to the right:

$$\gamma_2 = \frac{b_2 - r_{23} \cdot \gamma_3}{r_{22}}$$

Row 1: $r_{11} \cdot \gamma_1 + r_{12} \cdot \gamma_2 + r_{13} \cdot \gamma_3 = b_1$. Both γ_2 and γ_3 are known:

$$\gamma_1 = \frac{b_1 - r_{12} \cdot \gamma_2 - r_{13} \cdot \gamma_3}{r_{11}}$$

In general, for row i (working upward from $i = m$ to $i = 1$):

$$\gamma_i = \frac{b_i - \sum_{j=i+1}^m r_{ij} \cdot \gamma_j}{r_{ii}}$$

Each step divides by a diagonal entry r_{ii} . If any r_{ii} is near zero, the division blows up and γ becomes unreliable – this is the ill-conditioning that Step 4 monitors.

Once γ is found, it tells us how to combine the history. The same weights γ are applied to the iterate-change history ΔX in Step 5, steering the next update toward convergence faster than plain PCF.

Step 2

Factor ΔG into $Q \cdot R$ (QR decomposition)

$$\begin{array}{|c|c|c|} \hline \Delta g_{11} & \Delta g_{12} & \Delta g_{13} \\ \hline \Delta g_{21} & \Delta g_{22} & \Delta g_{23} \\ \hline \end{array} = \begin{array}{|c|c|} \hline q_{11} & q_{12} \\ \hline q_{21} & q_{22} \\ \hline \end{array} \cdot \begin{array}{|c|c|c|} \hline r_{11} & r_{12} & r_{13} \\ \hline 0 & r_{22} & r_{23} \\ \hline \end{array}$$

$(N_C \times m) = (2 \times 3) \quad (N_C \times N_C) = (2 \times 2) \quad (N_C \times m) = (2 \times 3)$

Step 3

Solve for weight vector γ using back-substitution

3a. Compute right-hand side: $b = Q^T \cdot \delta$

$$\begin{array}{|c|c|} \hline q_{11} & q_{21} \\ \hline q_{12} & q_{22} \\ \hline \end{array} \cdot \begin{array}{|c|} \hline \delta_1 \\ \hline \delta_2 \\ \hline \end{array} = \begin{array}{|c|} \hline b_1 \\ \hline b_2 \\ \hline \end{array}$$

$b = Q^T \cdot \delta$ projects the current correction δ into the rotated coordinate system

3b. Solve triangular system: $R \cdot \gamma = b$

$$\begin{array}{|c|c|c|} \hline r_{11} & r_{12} & r_{13} \\ \hline 0 & r_{22} & r_{23} \\ \hline \end{array} \cdot \begin{array}{|c|} \hline \gamma_1 \\ \hline \gamma_2 \\ \hline \gamma_3 \\ \hline \end{array} = \begin{array}{|c|} \hline b_1 \\ \hline b_2 \\ \hline \end{array}$$

$(m \times 1)$

Back-substitution (bottom-up, because R is triangular):
 $\gamma_2 = b_2 / r_{22}$ then $\gamma_1 = (b_1 - r_{12} \gamma_2 - r_{13} \gamma_3) / r_{11}$

Figure A4. Anderson Acceleration QR Decomposition and Solving for Weights: Q ($N_c \times N_c$): orthonormal matrix, its columns are perpendicular unit vectors. R ($N_c \times m$): upper triangular, zeros below the diagonal. The diagonal entries of R (r_{11}, r_{22}) measure how independent each column of ΔG is: large diagonal means that column adds new information; tiny diagonal means nearly parallel to others which means trouble (see column dropping). $b = Q^T \cdot \delta$ projects the current correction into the rotated coordinate system. Back-substitution (bottom-up, because R is triangular): no matrix inversion needed, just division and subtraction. The weights γ tell AA how much to trust each past iteration: large γ_j means that past step was very informative; $\gamma_j \approx 0$ reverts back to pure PCF.

A2.4 AA Step 4: Compute the Accelerated Update

A2.4a. The Anderson Acceleration update In order to compute the next iterate, we take a weighted average of all the past iterates and all the past corrections:

$$\omega^{(k+1)} = \omega^{\text{iterate}} + \omega^{\text{corrections}}$$

With $m = 3$ history columns, four weights $\alpha_0, \alpha_1, \alpha_2, \alpha_3$ define the next iterate as:

$$\begin{aligned} \omega^{(k+1)} &= \alpha_0 \omega^{(k-3)} + \alpha_1 \omega^{(k-2)} + \alpha_2 \omega^{(k-1)} + \alpha_3 \omega^{(k)} \\ &+ \beta_k \cdot (\alpha_0 \delta^{(k-3)} + \alpha_1 \delta^{(k-2)} + \alpha_2 \delta^{(k-1)} + \alpha_3 \delta^{(k)}) \end{aligned}$$

Thus, the new iterate is a blend of the recent iterates ($\omega^{\text{iterate}} = \sum \alpha_i \omega_j$) and the recent corrections ($\omega^{\text{corrections}} = \sum \alpha_i \delta_j$), scaled by the relaxation parameter β_k . The constraint $\sum \alpha_i = 1$ forces the blend to be a properly weighted average and one can express

$$\alpha_0 = \gamma_1$$

$$\alpha_1 = \gamma_2 - \gamma_1$$

$$\alpha_2 = \gamma_3 - \gamma_2$$

$$\alpha_3 = 1 - \gamma_3$$

where γ are the least-squares coefficients obtained by solving the minimization problem in Step 3 (Section A5.3).

A5.4c Substituting γ into the weighted-average-of-past-iterates term

The AA update formula (Awada et al. 2025) computes the next iterate as a weighted average of past iterates plus a weighted average of past corrections. We call the first part the “weighted past iterates” and the second part the “weighted past corrections.” Now we replace each α in the weighted-past-iterates term with its γ definition. We do this for $m = 3$ (four iterates), showing every term:

$$\begin{aligned} \omega^{\text{iterate}} &= \gamma_1 \omega^{(k-3)} + (\gamma_2 - \gamma_1) \omega^{(k-2)} + (\gamma_3 - \gamma_2) \omega^{(k-1)} + (1 - \gamma_3) \omega^{(k)} \\ &= \gamma_1 \omega^{(k-3)} + \gamma_2 \omega^{(k-2)} - \gamma_1 \omega^{(k-2)} + \gamma_3 \omega^{(k-1)} - \gamma_2 \omega^{(k-1)} + \omega^{(k)} - \gamma_3 \omega^{(k)} \end{aligned}$$

Now collect the terms that multiply each γ_i :

$$\omega^{\text{iterate}} = \omega^{(k)} + \gamma_1 (\omega^{(k-3)} - \omega^{(k-2)}) + \gamma_2 (\omega^{(k-2)} - \omega^{(k-1)}) + \gamma_3 (\omega^{(k-1)} - \omega^{(k)})$$

With $\Delta\omega^{(i)} = \omega^{(i+1)} - \omega^{(i)}$ (see Step 2).

$$\omega^{\text{iterate}} = \omega^{(k)} - \gamma_1 \Delta\omega^{(k-3)} - \gamma_2 \Delta\omega^{(k-2)} - \gamma_3 \Delta\omega^{(k-1)}$$

The three columns of ΔX are $\Delta\omega^{(k-3)}, \Delta\omega^{(k-2)}, \Delta\omega^{(k-1)}$. Multiplying each column by its γ and summing is the matrix-vector product $\Delta X \cdot \gamma$. So:

$$\omega^{\text{iterate}} = \omega^{(k)} - \Delta X \cdot \gamma$$

A2.4d Same Substitution for the Correction Sum The weighted-past-corrections term is $\alpha_0\delta^{(k-3)} + \alpha_1\delta^{(k-2)} + \alpha_2\delta^{(k-1)} + \alpha_3\delta^{(k)}$. We do the same substitution: replace each α with its γ definition, open parentheses, and collect terms.

The algebra is identical to A5.4c but with δ instead of ω . Each γ_j multiplies $\delta^{(i)} - \delta^{(i+1)} = -\Delta\delta^{(i)}$, and the leftover is $\delta^{(k)}$. So:

$$\omega^{\text{corrections}} = \delta^{(k)} - \gamma_1\Delta\delta^{(k-3)} - \gamma_2\Delta\delta^{(k-2)} - \gamma_3\Delta\delta^{(k-1)}$$

The columns of ΔG are $\Delta\delta^{(k-3)}, \Delta\delta^{(k-2)}, \Delta\delta^{(k-1)}$. So:

$$\omega^{\text{corrections}} = \delta^{(k)} - \Delta G \cdot \gamma$$

A2.4e Putting It Together The original formula was: $\omega^{(k+1)} = \omega^{\text{iterate}} + \beta_k \cdot \omega^{\text{corrections}}$. Substituting:

$$\omega^{(k+1)} = (\omega^{(k)} - \Delta X \cdot \gamma) + \beta_k(\delta^{(k)} - \Delta G \cdot \gamma)$$

This is the Anderson Acceleration update formula (Eq. 3 in Awada et al. 2025, rewritten in our notation). In the general Anderson acceleration framework, β_k controls how far the solver moves toward the next estimate at each iteration (Awada et al. 2025). If $\beta_k = 1$, the solver fully applies the correction it has computed which means it takes the complete step. If β_k is set to a smaller value (for example 0.5), the solver only applies half the correction which means it moves halfway and then re-evaluates before taking another step. In CompLaB3D, we took $\beta_k = 1$ because the equilibrium system is formulated using positive continued fractions (PCF). In the PCF formulation, species concentrations are expressed as fractions built from sums of formation constants and other positive quantities which means the structure of these expressions guarantees that the output of every iteration remains positive and bounded. This stability eliminates our need for partial steps, so the full correction ($\beta_k = 1$) can be safely applied at every iteration.

$$\omega^{(k+1)} = (\omega^{(k)} - \Delta X \cdot \gamma) + 1 \cdot (\delta^{(k)} - \Delta G \cdot \gamma)$$

Rearranging gives

$$\omega^{(k+1)} = \omega^{(k)} + \delta^{(k)} - (\Delta X + \Delta G) \cdot \gamma$$

$\omega^{(k)} + \delta^{(k)}$ is what Section A4 called the plain PCF update, i.e. $\omega_{\text{PCF}}^{(k)} = \omega^{(k)} + \delta^{(k)}$ so that

$$\omega^{(k+1)} = \omega_{\text{PCF}}^{(k)} - (\Delta X + \Delta G) \cdot \gamma$$

Step 4

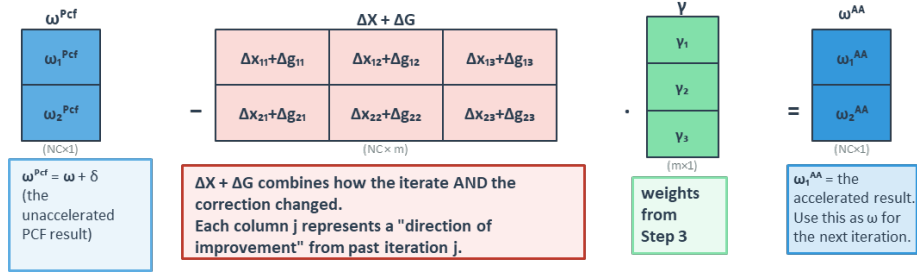


Figure A5. Anderson Acceleration: The Accelerated Update $\omega_{\text{AA}} = \omega_{\text{PCF}} - (\Delta X + \Delta G) \cdot \gamma$. Start with the pure PCF result (ω_{PCF}), but slowly. Subtract a “steering correction” $= (\Delta X + \Delta G) \cdot \gamma$: this uses history to predict a shortcut toward the solution. $\Delta X + \Delta G$ combines how both the iterate and the correction changed; each column j represents a direction of improvement from past iteration j . The result typically converges in small numbers of iterations instead of too many iterations.

References

- Awada, R., Carayrou, J., Rosier, C.: Anderson acceleration. Convergence analysis and applications to equilibrium chemistry. *Applied Numerical Mathematics*. 208, 60–75 (2025). <https://doi.org/10.1016/j.apnum.2024.01.022>
- Tadanier, C.J., Eick, M.J.: Formulating the charge-distribution multisite surface complexation model using FITEQL. *Soil Science Society of America Journal*. 66, 1505–1517 (2002). <https://doi.org/10.2136/sssaj2002.1505>